

Trabajo Práctico Integrador

Proceso automatizado: Solicitud de Vacaciones

Integrantes

García, Danely Yassmin

Rey, Martin Nicolas

Tecnicatura Universitaria en Programación a Distancia
Organización Empresarial

Docente Titular

Prof. Gabriela Martínez

Cátedra: Organización Empresarial
Organización ficticia: TechCorp S.A.
Plataforma del bot: Consola / Python

TABLA DE CONTENIDO

1. Introducción y descripción del proceso.....	3
2. Modelado BPMN 2.0.....	3
2.1 Diagrama AS-IS (proceso actual – manual).....	3
2.2 Diagrama TO-BE (proceso automatizado con chatbot).....	3
3. Perspectiva técnica.....	3
3.1 Plataforma y lenguaje.....	3
3.2 Herramientas de IA utilizadas.....	4
3.3 Persistencia – Base de datos simulada.....	4
3.4 Gestión de estados – Máquina de estados.....	4
4. Diccionario de datos.....	4
5. Manual de usuario.....	5
6. Pruebas – Caminos infelices (Unhappy Path).....	5
7. Repositorio de código y conclusión.....	6

1. INTRODUCCIÓN Y DESCRIPCIÓN DEL PROCESO

El presente trabajo integra los conocimientos de modelado de procesos de negocio (BPMN 2.0) con la implementación de un chatbot en Python que automatiza el proceso administrativo de solicitud de vacaciones dentro de la organización ficticia TechCorp S.A.

Actualmente este proceso se realiza de forma manual mediante formularios en papel o correo electrónico, lo que genera demoras, pérdida de información y falta de trazabilidad. La solución propuesta automatiza el flujo completo, desde la identificación del empleado hasta el registro del resultado, eliminando la intervención manual de RRHH.

2. MODELADO BPMN 2.0

2.1 DIAGRAMA AS-IS (PROCESO ACTUAL – MANUAL)

El proceso AS-IS involucra tres actores: el Empleado, el área de RRHH y el Jefe. El empleado completa un formulario y lo envía. RRHH revisa manualmente el saldo de días disponibles y, si es suficiente, lo eleva al jefe. El jefe decide y comunica el resultado. Este flujo tiene tiempos de espera de hasta 72 horas y no deja registro digital automático.

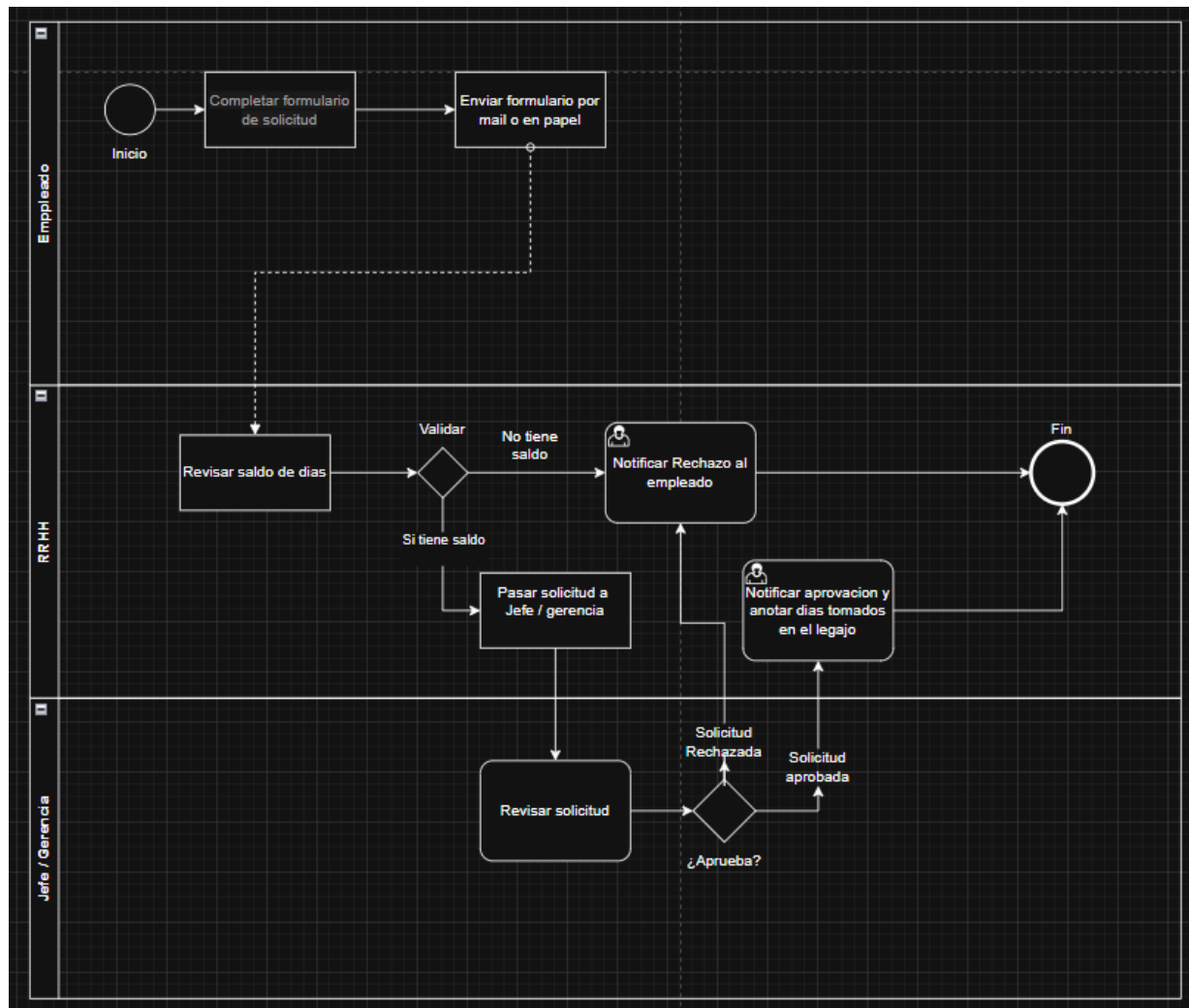


Figura 1: Diagrama BPMN AS-IS – proceso manual de solicitud de vacaciones.

2.2 DIAGRAMA TO-BE (PROCESO AUTOMATIZADO CON CHATBOT)

El proceso TO-BE reemplaza el carril de RRHH por el carril Sistema/Bot. El chatbot ejecuta las siguientes tareas de servicio de forma automática:

- Validación del empleado contra la base de datos simulada (Gateway 1).
- Consulta del saldo de días disponibles (Gateway 2).
- Registro de la solicitud
- Comprobación de la solicitud y actualización del saldo (Gateway 3).

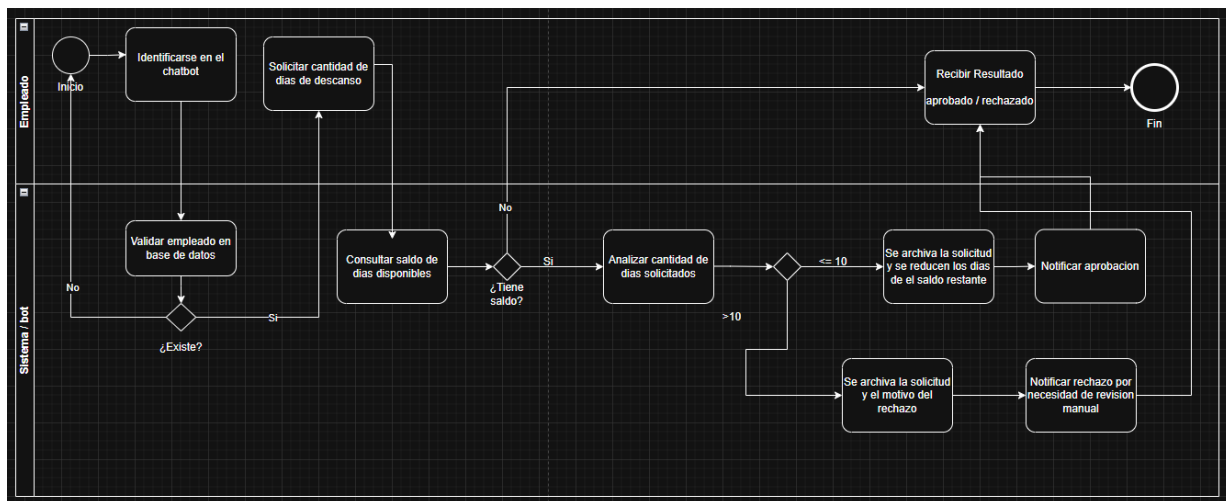


Figura 2: Diagrama BPMN TO-BE – proceso automatizado

3. PERSPECTIVA TÉCNICA

3.1 PLATAFORMA Y LENGUAJE

Se optó por Python 3 con interfaz de consola, por ser el lenguaje trabajado durante la cursada y por permitir una simulación clara del flujo sin dependencias externas. En un entorno productivo, el mismo bot podría migrarse a Telegram usando la librería python-telegram-bot o a una interfaz web con Flask.

3.2 HERRAMIENTAS DE IA UTILIZADAS

Se utilizó Claude (Anthropic) como asistente de IA para: diseñar el flujo del proceso, sugerir la estructura de la máquina de estados, generar el esqueleto del código Python y revisar los caminos de error. Los prompts utilizados incluyeron consultas como: “¿Cómo implemento una máquina de estados en Python para un chatbot de consola?” y “¿Qué gateways necesito para modelar una solicitud de vacaciones en BPMN 2.0?”

3.3 PERSISTENCIA – BASE DE DATOS SIMULADA

La base de datos se implementa como un diccionario Python que simula una tabla de empleados. Cada registro contiene: legajo (clave), nombre, puesto y días disponibles. Las solicitudes aprobadas o rechazadas se almacenan en una lista solicitudes_registradas. En producción, esta estructura se reemplazaría por una base de datos SQLite o PostgreSQL.

3.4 GESTIÓN DE ESTADOS – MÁQUINA DE ESTADOS

El bot implementa una máquina de estados mediante la variable estado_actual, que registra en qué paso del proceso se encuentra el usuario:

Estado	Descripción
INICIO	Esperando identificación del empleado
IDENTIFICADO	Empleado verificado, esperando cantidad de días
DIAS_INGRESADOS	Días ingresados, verificando saldo disponible
APROBACION_JEFE	Solicitud enviada, pendiente de decisión del jefe
FINALIZADO	Proceso terminado, resultado registrado

Tabla 1: Máquina de estados del chatbot.

4. DICCIONARIO DE DATOS

Variable	Tipo	Descripción
empleados	dict	Base de datos simulada. Clave: legajo. Valores: nombre, puesto, dias_disponibles.
legajo	str	Identificador único del empleado. Formato: E001, E002... Clave primaria.
nombre	str	Nombre completo del empleado.
puesto	str	Cargo o rol del empleado dentro de la organización.
dias_disponibles	int	Cantidad de días de vacaciones disponibles. Se descuenta al aprobar.
dias_pedidos	int	Cantidad de días solicitados por el empleado en la sesión actual.
estado_actual	str	Estado actual en la máquina de estados del proceso.
solicitudes_registradas	list[dict]	Registro de todas las solicitudes procesadas en la sesión (aprobadas y rechazadas).
aprobado	bool	Resultado de la decisión del jefe: True = aprobado, False = rechazado.

Tabla 2: Diccionario de datos del sistema.

5. MANUAL DE USUARIO

Para ejecutar el chatbot, correr el archivo chatbot_vacaciones.py con Python 3:

```
python chatbot_vacaciones.py
```

Comandos disponibles durante la sesión:

- salir → cancela el proceso en cualquier momento.
- [legajo] → ingresá tu legajo cuando el bot lo pida (ej: E001).
- [número] → ingresá la cantidad de días solicitados como número entero positivo.
- s / n → al finalizar, confirmá si querés iniciar otro proceso.

Empleados de prueba cargados en el sistema:

Legajo	Nombre	Días disponibles	Resultado esperado
E001	Ana García	15	Aprobado si solicita ≤ 10 días
E002	Carlos López	5	Aprobado si solicita ≤ 5 días
E003	María Torres	0	Rechazado siempre (saldo 0)
E004	Luis Pérez	20	Aprobado si solicita ≤ 10 días

Tabla 3: Empleados de prueba y resultados esperados.

6. PRUEBAS – CAMINOS INFELICES (UNHAPPY PATH)

El sistema maneja los siguientes errores de entrada, cubriendo todos los caminos alternativos del diagrama BPMN:

Legajo vacío

Entrada: El usuario presiona Enter sin escribir nada.

Respuesta del bot: El bot avisa que el legajo no puede estar vacío y permite reintentar.

Legajo inexistente

Entrada: El usuario ingresa un legajo que no existe en el sistema (ej: E999).

Respuesta del bot: El bot informa que no fue encontrado. Después de 3 intentos, cierra la sesión.

Cantidad no numérica

Entrada: El usuario ingresa texto en lugar de un número (ej: 'cinco').

Respuesta del bot: El bot informa que el valor no es válido y solicita un número entero.

Cantidad negativa o cero

Entrada: El usuario ingresa 0 o un número negativo.

Respuesta del bot: El bot rechaza la entrada y explica que debe ser un número positivo.

Saldo insuficiente

Entrada: El empleado solicita más días de los que tiene disponibles.

Respuesta del bot: El bot informa el saldo real, sugiere la cantidad máxima y cierra el proceso.

Saldo cero

Entrada: El empleado tiene 0 días disponibles (ej: E003).

Respuesta del bot: El bot informa que no hay días disponibles en el período actual.

Revision manual requerida

Entrada: El empleado solicita más de 10 días.

Respuesta del bot: El bot informa el rechazo y deriva al empleado a contactar a RRHH personalmente.

Comando 'salir'

Entrada: El usuario escribe 'salir' en cualquier momento.

Respuesta del bot: El bot cancela el proceso de forma limpia sin errores.

7. REPOSITORIO DE CÓDIGO Y CONCLUSIÓN

REPOSITORIO GITHUB

El código fuente del chatbot, el README.md con instrucciones de despliegue y los diagramas BPMN exportados se encuentran disponibles en el repositorio:

https://github.com/Blue210-dev/TPI_OE

CONCLUSIÓN

Este trabajo integró el modelado de procesos con BPMN 2.0 y la programación en Python para automatizar un proceso administrativo real. El chatbot desarrollado cubre el flujo completo de solicitud de vacaciones, implementa una máquina de estados, maneja errores de entrada (camino infeliz) e interactúa con una base de datos simulada.

La coherencia entre el diagrama TO-BE y el código garantiza que cada gateway del BPMN tenga su correspondiente estructura lógica en Python (if/else), cumpliendo con el objetivo central del trabajo práctico integrador.